Team Sentinel

The Account That Appeared Twice

Visibility Assessment and Telemetry Evaluation, SOC Analyst Bootcamp Week 2

October 2026 · Scenario 1: Suspicious User Creation · Scenario 2: Privilege Escalation

1. Executive Summary

It began with a name that looked harmless. On a Windows laptop and on a Kali machine in our lab, someone created an account called `svc_backup`. It sounds like something IT would set up, so nobody would look twice. Then the account was given power: administrator rights on Windows, sudo rights on Linux. Then it used that power. On Windows it installed a service. On Linux it read the password file and tripped a failed sudo attempt.

Our job was to play the analyst the morning after: looking only at the logs, can we tell this story?

Our answer is PARTIALLY. We can tell most of the story on each machine. We cannot prove that the same person was behind both.

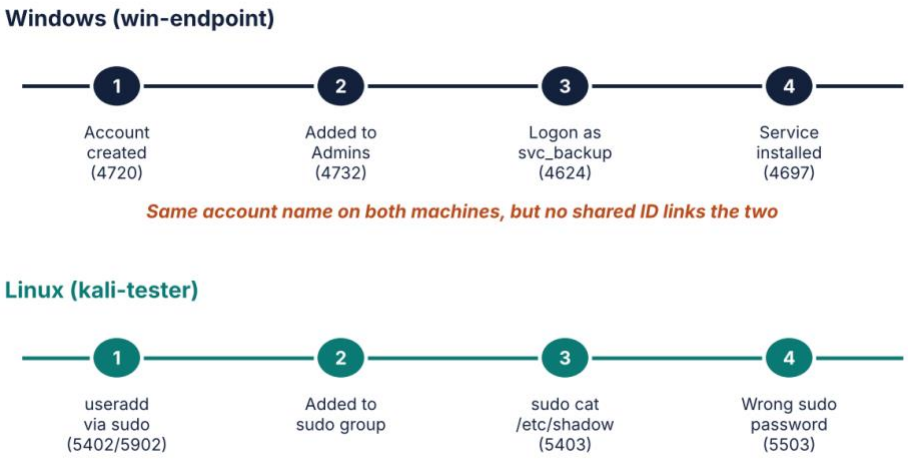


Figure 1. *The two parallel stories we had to reconstruct. The numbers are the order of events on each host; the event IDs and Wazuh rule numbers are the log entries that told us.*

What the logs gave us easily. Windows recorded that the account was created, who created it, and that it joined the Administrators group. Linux recorded the new user, the new group, and every command that went through sudo.

What they did not give us until we added more tools. Default Windows logs never recorded the command that was typed. Default Linux logs only recorded commands run through sudo. Once we added Sysmon on Windows and Auditd on Linux, those blanks filled in. But Auditd on its own stayed silent in Wazuh until we wrote three custom rules, and one Windows service event (4697) never showed up in the dashboard search even though it was in the Windows log.

A warning we found along the way. Better logging has a price. Sysmon recorded a password in clear text, because it had been typed on the command line.

How sure are we? Very sure inside each machine. Not sure across the two, because the only thing tying them together is a name we chose ourselves.

2. Incident Scope and Questions

Two scenarios, two machines, one account name. Here is what we set out to look at.

Scenario	What we did	Windows (win-endpoint)	Linux (kali-tester)

1. Suspicious user creation	Create a local user and give it elevated rights	`svc_backup` created and added to Administrators (`svc_backup2` in the Sysmon run)	`svc_backup` created and added to the `sudo` group (`svc_backup2` in the Auditd run)
2. Privilege escalation	Use the new rights for things a normal user cannot do	Logged on as `svc_backup`; installed service `testsvc3` pointing at `cmd.exe`	`sudo -l`, `sudo cat /etc/shadow`, and one failed sudo attempt

The questions we asked of the logs:

1. Are the two scenarios connected, or just two separate events?
2. What happened first, once we account for clocks that disagree?
3. What can we prove, and what are we only guessing?
4. What is missing, and what would fill the gap?
5. Can we give management a straight answer: Yes, No or Partially?

Assumptions. The three machines share one NAT Network (the same one we used for last week's project), so they can reach each other and the network is not cut off from anything else. Every action was done on purpose by our team, as a test. We picked the name `svc_backup` ourselves for both machines.

Left out. We did not generate network connections or file-integrity events, so those areas are outside this report.

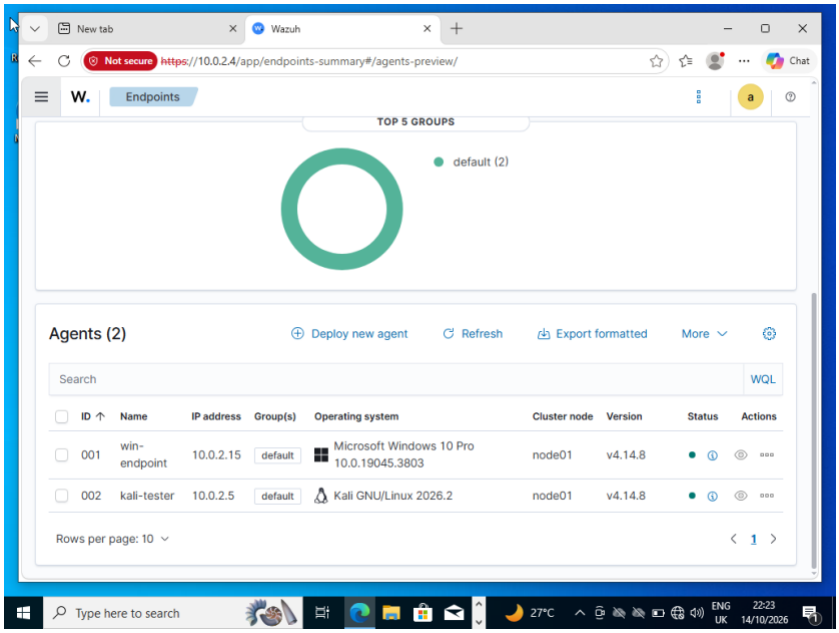


Fig 2. *Wazuh agents page showing both windows endpoints and kali-tester active*

3. Lab Architecture

Three virtual machines share one NAT network. The Ubuntu machine runs Wazuh and listens. The Windows and Kali machines each run a Wazuh agent that sends their logs over to it. Each agent also has one extra tool of its own: Sysmon on Windows and Auditd on Kali.

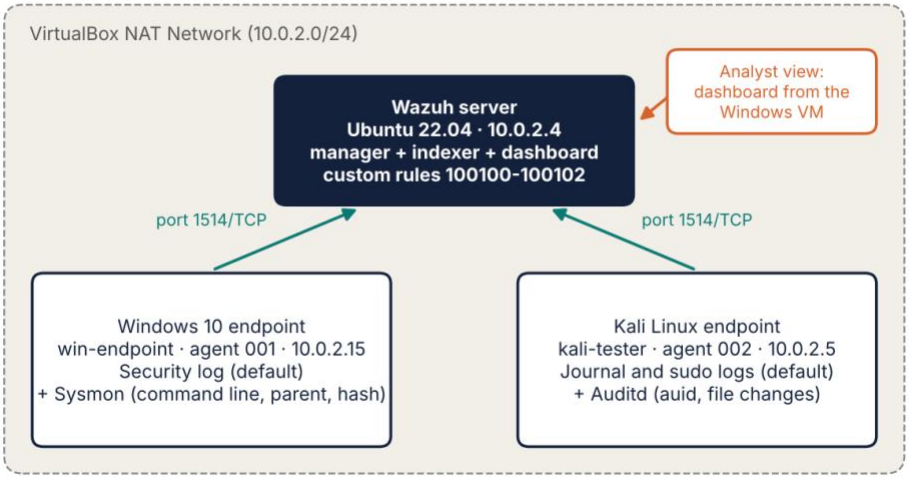


Figure 3. *Lab layout: one Wazuh server (Ubuntu), two agents, and the extra telemetry tool on each agent.*

Machine	Role	Address	Telemetry
Ubuntu 22.04 (4 GB RAM, 2 CPUs)	Wazuh 4.14.8 manager, indexer, dashboard	10.0.2.4	Custom rules 100100 to 100102
Windows 10, `win-endpoint` (agent 001)	Endpoint	10.0.2.15	Security log, Sysmon
Kali Linux, `kali-tester` (agent 002)	Tester and Linux endpoint	10.0.2.5	Journal and sudo logs, Auditd with six rules

A practical detail: the laptop that hosts the VMs cannot reach them directly through the NAT network, so we opened the dashboard from inside the Windows VM at `https://10.0.2.4`.

A note on clocks. The three machines do not agree on the time. Ubuntu runs in UTC, Windows is set to Pacific time, Kali is set to New York time with NTP switched off, and the dashboard shows times in the browser's own zone. We converted everything to UTC before comparing, and we trusted the Sysmon `UtcTime` field and Kali's UTC line most.

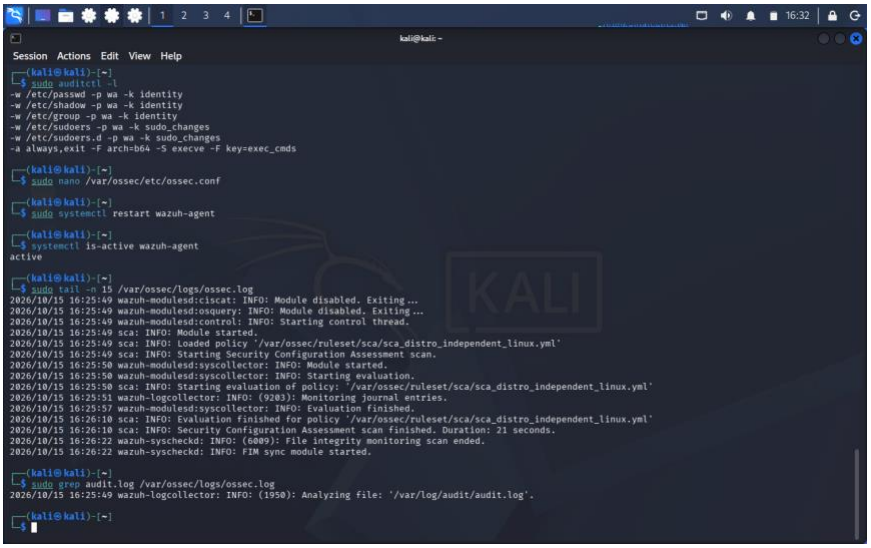


Figure 4. *Kali after restarting its Wazuh agent: the service is active and the log shows it monitoring the journal.*

4. Methodology

We ran every scenario twice on each machine. The first run used default logging only, the way a typical company would have it out of the box. The second run added our extra tool. That way we could compare the same action seen two ways.

1. **Windows, default logging.** Turned on audit policies for account management, process creation and group changes, then created `svc_backup` and made it an administrator.
2. **Windows, with Sysmon.** Installed Sysmon, forwarded its channel to Wazuh, and repeated the activity with a fresh account, `svc_backup2`.
3. **Windows, scenario 2.** Signed in as `svc_backup` and installed a test service. We switched on the "Security System Extension" audit setting first so service installs (event 4697) would be recorded.
4. **Linux, default logging.** Created `svc_backup`, added it to sudo, and used `sudo -l`, `sudo cat /etc/shadow` and one wrong password.
5. **Linux, with Auditd.** Installed Auditd, loaded six rules, forwarded the audit log, and repeated the activity with `svc_backup2`.
6. **Wazuh rules.** Added three custom rules so Wazuh would alert on the Auditd keys, then repeated a `usermod` and a `cat /etc/shadow` to generate alerts.

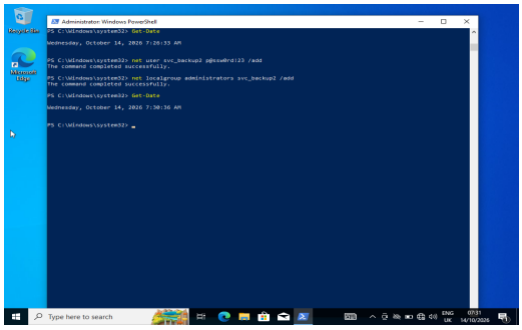


Figure 5. *Windows PowerShell: creating svc_backup2 and adding it to Administrators, with Get-Date before and after to time-stamp the run.*



Figure 6. *Kali: id, getent passwd and grep sudo /etc/group confirming svc_backup exists and is in the sudo group.*

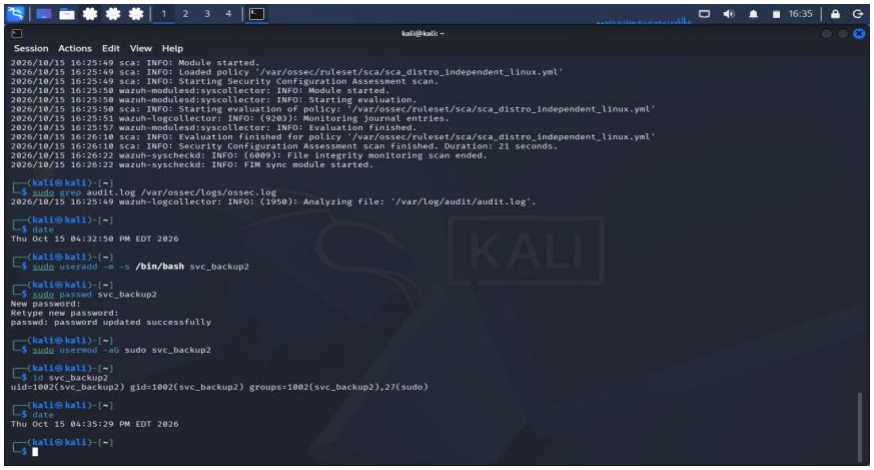


Figure 7. *auditctl -l on Kali listing the six Auditd rules we loaded (watches on passwd, shadow, group, sudoers and every execve).*

How we compared. For each action we asked the same four questions of each log source: who did it, what command ran, which process ran it, and what file or permission changed. The answers are in Sections 6 and 8.

5. Timeline Reconstruction

Here is the story in order. Times are in UTC. Windows shows its local time in some screenshots; it is set to Pacific time, which in October is UTC minus 7, and we confirmed that against Sysmon's own UTC stamp.

#	When (UTC)	Host	What happened	Where we saw it
1	14 Oct, about 02:14 (19:14 on the Windows clock)	Windows	`svc_backup` created (4720) by Glow Ree	Security log, Wazuh
2	Same session	Windows	`svc_backup` added to Administrators (4732, rule 60154)	Security log, Wazuh
3	Same session	Windows	`net.exe` / `net1.exe` process created (4688), no command line	Security log
4	14 Oct, about 14:26	Windows	`net user svc_backup2 ... /add`, full command line (Sysmon Event 1)	Sysmon, rules 92031/92033
5	14 Oct, 7:26 (Windows clock)	Windows	Logon as `svc_backup` (4624, logon ID 0x1e3fc5)	Security log
6	After the logon above	Windows	Service `testsvc3` installed in that same logon (4697)	Security log (not in Wazuh search)
7	14 Oct, 21:29	Linux	`useradd svc_backup` run through sudo, new user and group (5402, 5902, 5901)	auth log, Wazuh
8	14 Oct,	Linux	First sudo use and `cat /etc/shadow` (5403)	auth log, Wazuh
9	14 Oct,	Linux	Wrong sudo password (5503)	PAM log, Wazuh
10	15 Oct, 20:33	Linux	`useradd svc_backup2` recorded with `auid`	Auditd
11	15 Oct, 15:12:31	Linux	`usermod` and `cat /etc/shadow` recorded; alerts only after custom rules	Auditd, rules 100100/100102

The default Linux run and the Auditd run happened on different days, which is why the Linux times above are a day apart.

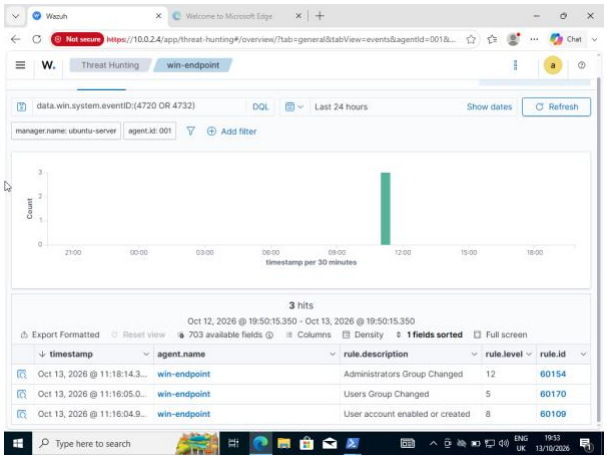


Figure 8. *Wazuh search for events 4720 and 4732: the user-created and Administrators-group alerts for win-endpoint.*

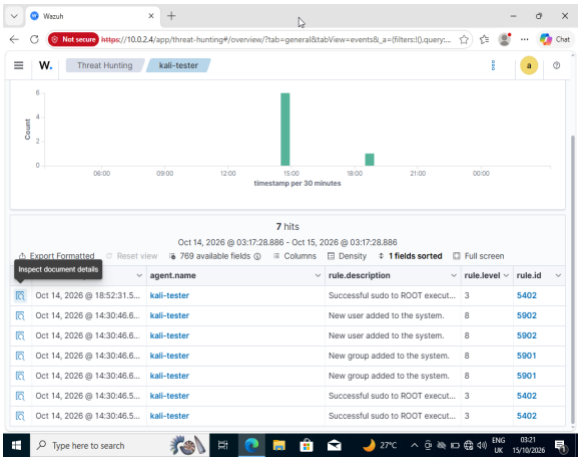


Figure 9. *Wazuh showing 7 hits for kali-tester: new user (5902), new group (5901) and sudo to root (5402).*

6. Telemetry Analysis

6.1 Windows with default logging

The default Security log told us a lot. Event 4720 says the account `svc_backup` was created, and 4732 says it was added to the Administrators group. Both name Glow Ree as the person who did it. Event 4688 says a `net` process ran, but it does not tell us what was typed after it. For service installs, 4697 shows the service name and file, but it does not say who created the service.

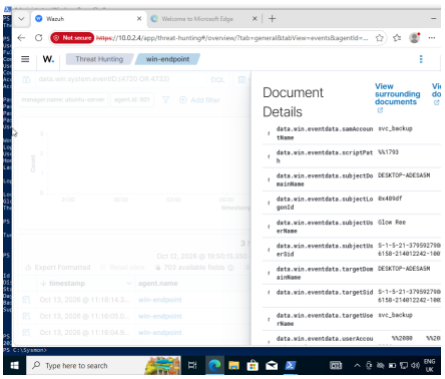


Figure 10. *Event 4720: account name svc_backup.*

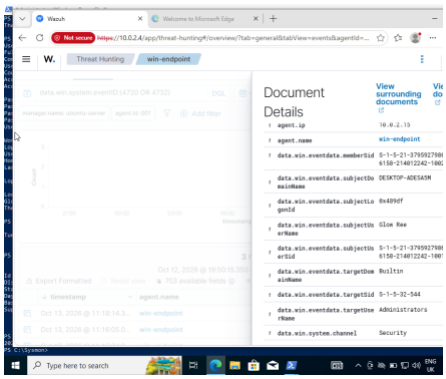


Figure 11. *Event 4732: by Glow Ree, group S-1-5-32-544 (Administrators).*

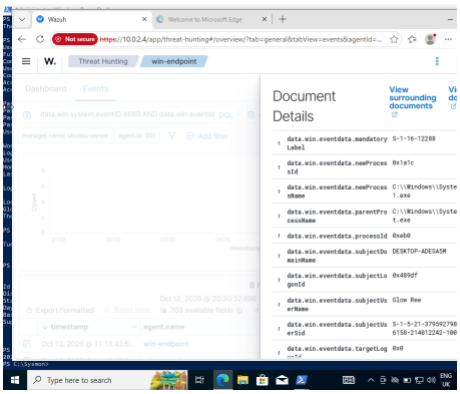


Figure 12. *Event 4688: net1.exe started by net.exe, with no command line field.*

6.2 Windows with Sysmon

Sysmon Event 1 filled the gap. For each process it recorded the full command line, the parent process, the user, the integrity level, the file hashes and a precise UTC time. Wazuh's Sysmon rules (92031, 92033, 92039) fired on the activity as well. This is where "who made the account" turned into "who typed exactly this command".

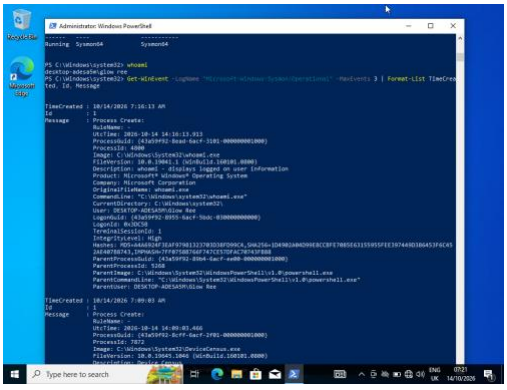


Figure 13. *Sysmon Event 1 for whoami: command line, hashes, integrity level, PowerShell parent.*

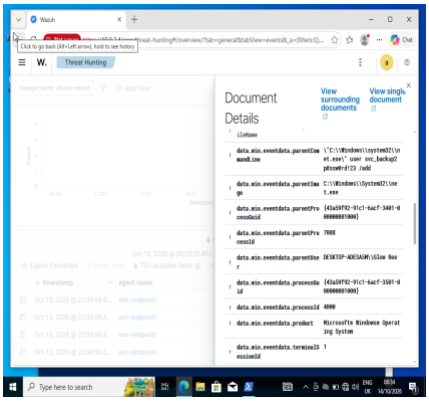


Figure 14. *Parent command line net.exe user svc_backup2 ... /add; the lab password is stored in clear text.*

6.3 Linux with default logging

Linux told us the new user, the new group and the sudo activity, but only because the commands went through sudo. A command typed in a plain shell would have left no trace of itself.

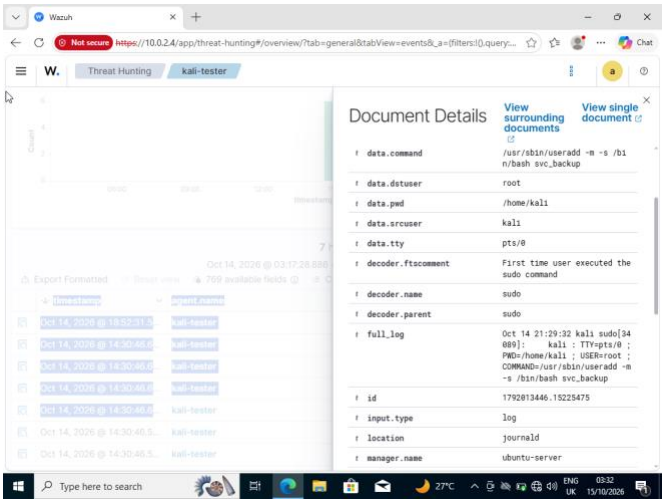


Figure 15. *Rule 5402: data.command is /usr/sbin/useradd -m -s /bin/bash svc_backup, run by user kali as root.*

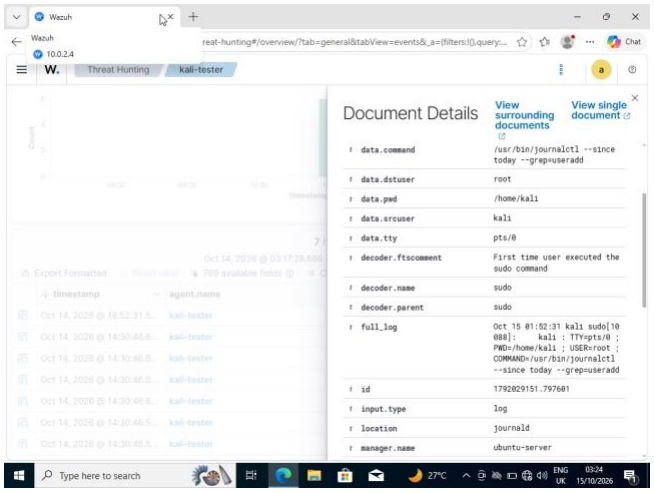


Figure 16. *Another 5402: /usr/bin/journalctl --since today --grep useradd, showing how we looked back at our own activity.*

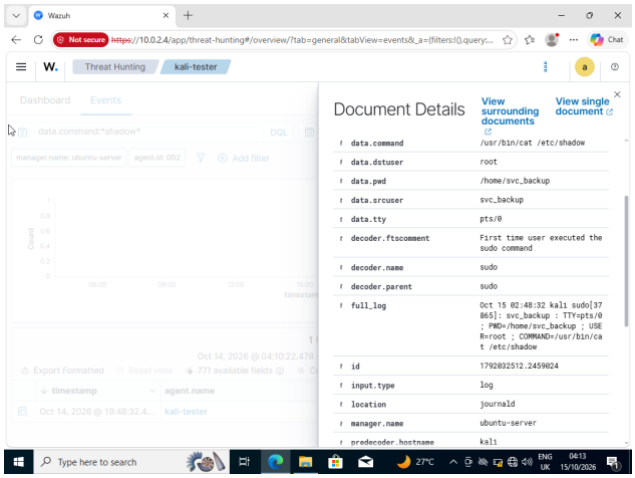


Figure 17. *Kali alert list in Wazuh: PAM session and sudo alerts (5402, 5403, 5501, 5502, 5503).*

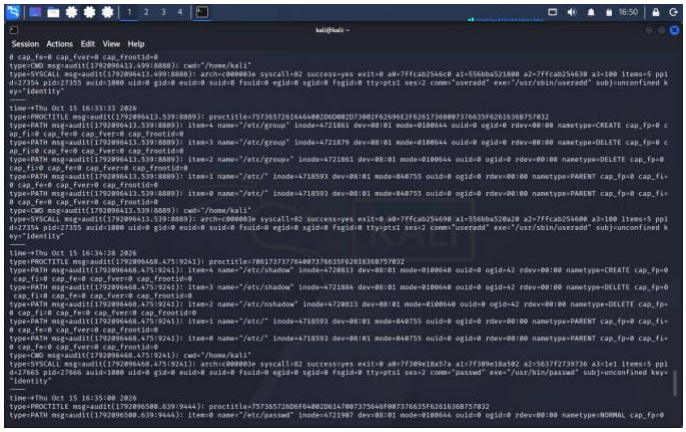


Figure 18. *Raw ausearch output on Kali: useradd records with file paths and the original user ID.*

6.4 Linux with Auditd

Auditd recorded who really did it. Each record carries the audit user ID (`auid`) that stays the same even after someone switches user, plus the program, terminal and exact file touched. It saw changes to `/etc/passwd`, `/etc/group` and `/etc/shadow`. But Wazuh only keeps events that match a rule, so none of this appeared in the dashboard until we wrote three custom rules. Also, our watches were set to write and attribute changes (`-p wa`), which means a plain `cat /etc/shadow` was not captured by them.

Source	Best at	Weakest at
Windows default	Who, and account or group changes	Command line, process detail
Sysmon	Command, parent process, hashes	Not a permission record; exposes secrets
Linux default	Sudo use and user creation	Commands outside sudo, file detail
Auditd	Real actor and exact file change	Needs custom Wazuh rules and tuning

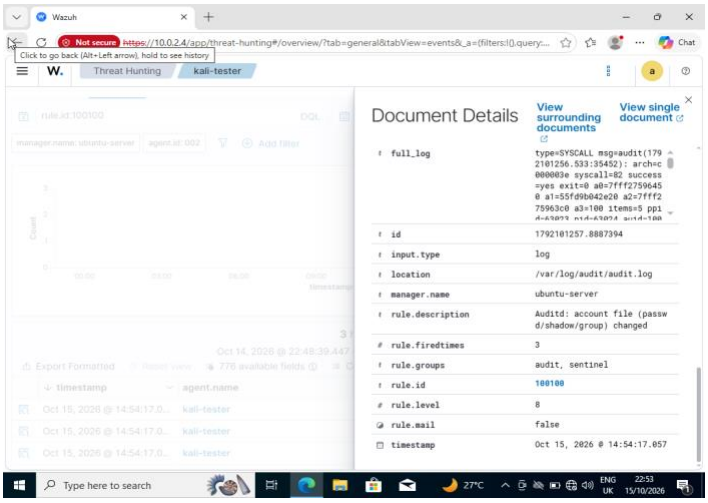


Figure 19. *Wazuh alert for rule 100100*

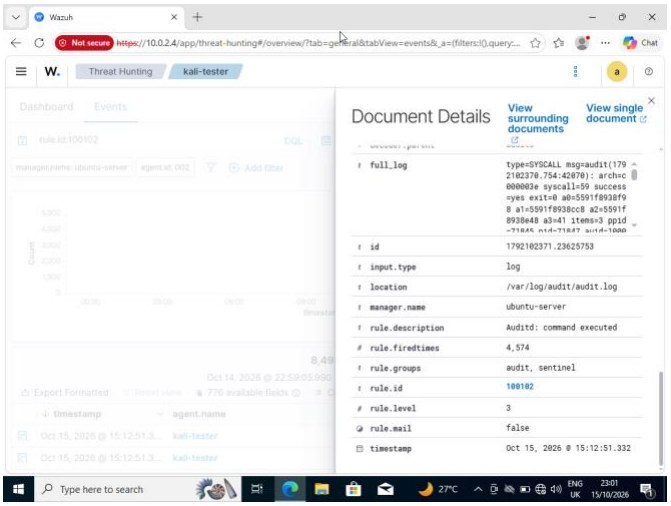


Fig 20. *Wazuh alert for rule 100102*

7. Correlation Analysis

Telling a story means tying events together. This is how well we could do it.

Inside Windows, the thread is strong. Windows gives every logon a number. Logon ID `0x1e3fc5` appears on the logon of `svc_backup` (event 4624, as the linked logon) and again on the 4697 event that installed `testsvc3`. So we can say, with evidence, that the service was installed from that very session. Sysmon adds a second thread: each `net` command points back to the PowerShell window of the person who ran it.

Inside Linux, the thread is strong too. The sudo user and the `auid` follow a person through every command, even after they switch to root.

Between the two machines, the thread is thin. Nothing in either log refers to the other machine. What joins the stories is the name `svc_backup`, which we picked ourselves, and the order of events. A real intruder would likely use a different name on each machine, and that link would vanish. The clock differences make the order uncertain as well.

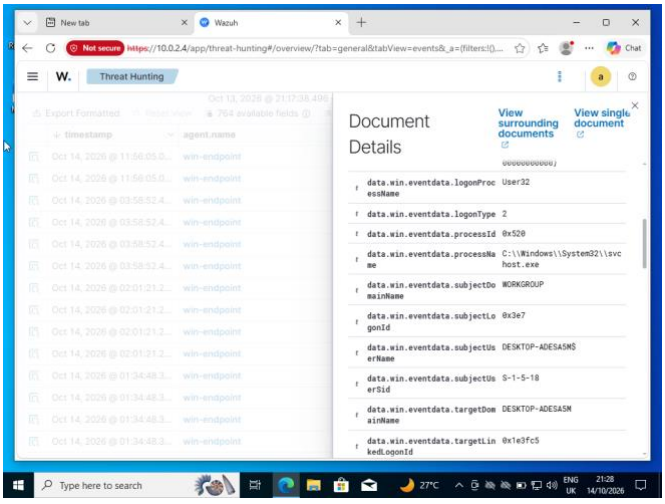


Figure 21. *Windows 4624 logon: logon type 2 and target linked logon ID 0x1e3fc5.*

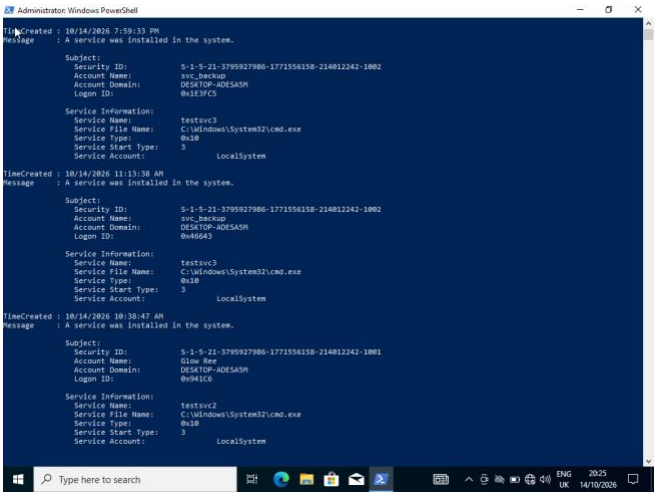


Figure 22. *Windows 4697 service installs: the entry for testsvc3 by svc_backup carries Logon ID 0x1E3FC5.*

Link	Fields used	Strength

Account creation to admin rights (Windows)	Account name, 4720 and 4732	Strong
Logon to service install (Windows)	Logon ID	Strong
Command to process (Windows)	Sysmon parent image and user	Strong
User to command (Linux)	sudo user, `auid`	Strong
Windows to Linux	Account name, sequence	Weak

8. Visibility Comparison

This is the whole experiment on one page. Each cell says whether the logs could answer the question. "Not tested" means we did not run an activity that would have produced that evidence, so we cannot say yes or no.

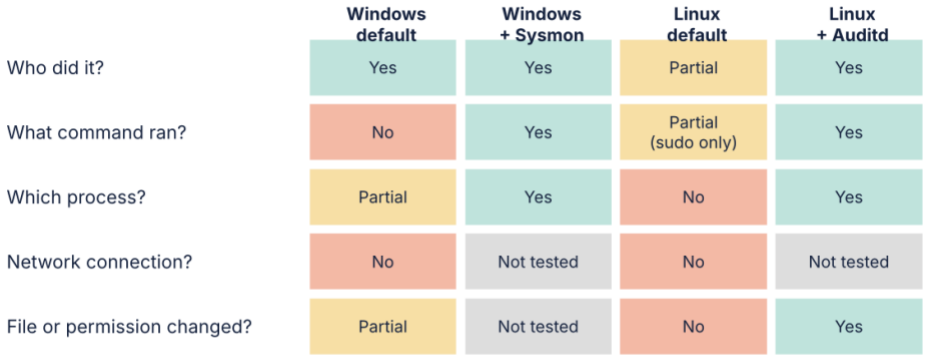


Figure 24. *Visibility matrix for the four setups. Every cell has a written label as well as a colour.*

What stands out. Default logging is good at "who" and weak at "what exactly". Sysmon fixes the command and process gap on Windows. Auditd fixes the real-actor and file gap on Linux.

9. Visibility Gap Analysis

These are the places where the story had holes.

#	Gap	What we saw	Why it matters	Fix
1	Windows command missing in default 4688	No command line field	Cannot tell which account was made	Command-line auditing or Sysmon
2	Service installer not named; 4697 not in Wazuh search	4697 has no creator; no dashboard hit	Service persistence is hard to attribute	Enable the audit setting, add a Wazuh rule, match by logon ID
3	Linux actor unclear in default logs	Only sudo lines name the user	Anything outside sudo is invisible	Auditd and `auid`
4	Auditd data silent in Wazuh	No alerts until rules 100100 to 100102	Collected but never reviewed	Rule for every audit key
5	Reads of `/etc/shadow` missed by a write-only watch	`-p wa` ignored `cat`	Credential reading goes unseen	Add `r` to the watch
6	Clocks disagree	Pacific, New York and UTC; Kali NTP off	Cross-host order is unreliable	NTP everywhere, in UTC
7	Sysmon stores passwords	Password visible in `commandLine`	The log becomes a target	Restrict access, mask fields

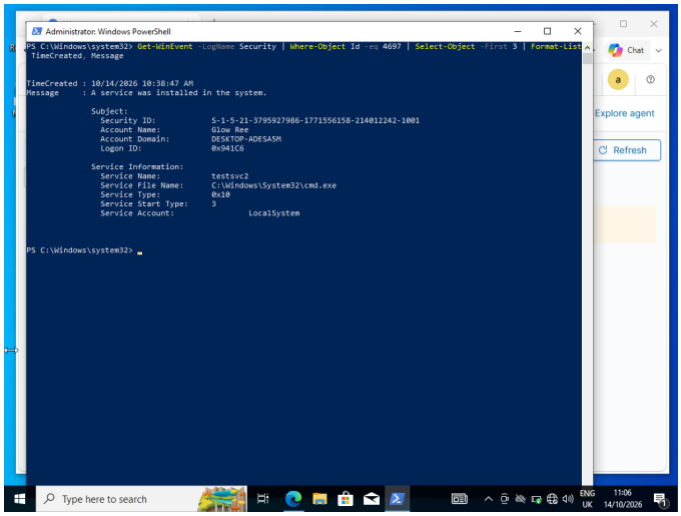


Figure 23. *Windows Event 4697 query: many service installs appear, mostly system noise, and none names the creating user as the actor we want.*



Figure 24. *Kali: NTP inactive and the clock not synchronized, the root of gap 6.*

10. Findings and Lessons Learned

What we found

1. Default Windows logs prove an account was made and elevated, but not how.
2. Sysmon turns "a process ran" into "this exact command, from this parent, by this user".
3. Linux defaults are only strong when sudo is used; Auditd records the real person and the exact file.
4. Collecting a log is not the same as seeing it: Auditd data sat unused until we wrote rules.
5. The thread between events is strong inside a host and weak between hosts.
6. Better telemetry brings new risks, such as passwords stored in clear text.

What we would tell the next team

- Turn on the audit setting first, then run the test, or the evidence will not exist.
- Check the clocks before you trust any timeline.
- Keep an action log with exact commands and UTC times from the first minute.
- Confirm which VM you are on before you run a command. We lost time running things on the wrong machine.
- Test your detections by running the activity and searching for it; never assume a rule exists.

11. Recommendations

If we were advising the company, this is the order we would do things in.

Priority	Action	Closes gap	Effort
High	Deploy Sysmon with a tuned configuration on all Windows endpoints and forward it to Wazuh	1	Medium
High	Write Wazuh rules for every Auditd key (identity, sudo_changes, exec_cmds)	4	Low
High	Sync every host to NTP and standardise on UTC	6	Low
Medium	Turn on Security System Extension auditing and alert on 4697 and 7045	2	Low
Medium	Extend Auditd watches to cover reads of `/etc/shadow`	5	Low
Medium	Roll Auditd out to every Linux host and alert on `auid` mismatches	3	Medium
Medium	Restrict access to Sysmon data and mask passwords in command lines	7	Medium
Low	Add a shared investigation tag so events on different hosts can be tied together	Section 7	High

12. Conclusion and Management Brief



Figure 27. *Our answer to the assignment question.*

Our answer is PARTIALLY. Here is the reasoning in plain terms.

- **Why not YES.** With default logging we could see that an account appeared, became powerful and was used. But we could not see the Windows commands, and we could not see who ran Linux commands outside sudo. And across the two machines, nothing in the logs ties the two stories together except a name we chose.
- **Why not NO.** Inside each machine we could rebuild the story with real evidence: the logon ID on Windows, `auid` and sudo on Linux. With Sysmon and Auditd added, the detail was strong.
- **So PARTIALLY.** Each machine's story can be told well once the right tools are on. The whole incident, across both machines, cannot be proven from the logs alone.

For management

- Default logging gives a headline, not the full story.
- Two inexpensive additions, Sysmon and Auditd, supply most of the missing detail.
- Collected data does nothing without detection rules.
- Synchronised clocks are a basic requirement for any investigation.
- Detailed logs need protecting, because they can contain passwords.

Decision requested: approve the Sysmon and Auditd rollout, the custom Wazuh rules and NTP standardisation.